



11.5. B-Trees

"When one node is not enough, let the tree store more."

Previous Sections:

 [11.3. AVL Trees](#)

 [11.4. Red-Black Trees](#)

Contents:

[1. What is a B-Tree?](#)

[1.1. Visualizing a B-Tree](#)

[1.2. Properties of a B-Tree](#)

[2. Operations on B-Tree](#)

[3. Implementation of B-Tree](#)

[Summary](#)

[References](#)

Before we introduce the B-Tree, we should first ask a simple question: ***Why do we need another tree structure when we already have Binary Search Trees?***

A Binary Search Tree works well when the amount of data is relatively small. Each node stores a single key and can have at most two children. The problem begins when the amount of data grows into millions of records stored on disks, databases, or file systems.

Imagine searching for a file inside a huge storage system. Every time we move down one level of a tree, the system may need to access another block of data from disk. If the tree becomes tall, the number of disk accesses increases, and the search becomes slower.

This is where the B-Tree steps forward. Instead of storing only one key per node, a B-Tree stores multiple keys inside the same node. By doing so, a single node can guide us toward many different branches at once. The result is a much shorter tree, fewer disk accesses, and significantly better performance for large-scale storage systems.

In other words, while a Binary Search Tree grows upward by adding more levels, a B-Tree grows outward by storing more information inside each node.

1. What is a B-Tree?

A **B-Tree (Balanced Tree)** is a self-balancing search tree that stores sorted data and supports searching, retrieval, insertion, and deletion, all with logarithmic time complexity.

Unlike a Binary Search Tree, where each node has at most two children, a B-Tree allows each node to have multiple children. This design keeps the height of the tree very small even when storing a massive amount of data.

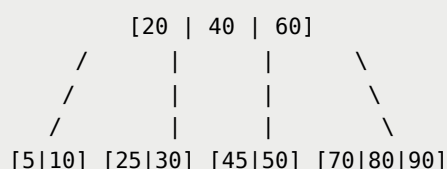
Because of this property, B-Trees are widely used in:

- Database Management Systems (DBMS)
- File Systems
- Storage Engines
- Disk-based indexing structures

The fundamental idea is simple: ***Instead of spreading keys across many levels, we group multiple keys together inside a node.***

1.1. Visualizing a B-Tree

Consider the following B-Tree:



Notice something interesting.

- The root node stores three keys: `[20 | 40 | 60]`
- These keys divide the data into four regions:
 - `Values < 20`
 - `20 < Values < 40`
 - `40 < Values < 60`
 - `Values > 60`

Therefore, the root has four child nodes. This is the core difference between a Binary Search Tree and a B-Tree.

- A Binary Search Tree asks: Left or Right?
- A B-Tree asks: Which range should we follow? One comparison can eliminate multiple possibilities at once.

Another Example



- The node: `[10|20]` contains two keys. Therefore it has $2 + 1 = 3$ children
- Similarly: `[40|50|60]` contains three keys. Therefore it has: $3 + 1 = 4$ children

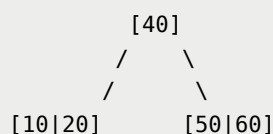
This relationship is always true in a B-Tree:

$$\text{Number of Children} = \text{Number of Keys} + 1$$

1.2. Properties of a B-Tree

For a B-Tree of order **N**:

- **Keys inside a node are sorted:** Each node contains **k keys** arranged in ascending order.
 - For example, `[10 | 25 | 40 | 55]`.
 - Since the keys are sorted, searching can quickly determine which child branch should be followed.
- **A node with **k** keys has **k + 1** children.**
 - For example, node `[20 | 40]` contains: 2 keys. Therefore, it must have 3 children
- **Every node has a maximum capacity**
 - For a B-Tree of order **N**: `Maximum children = N` and `Maximum keys = N - 1`
 - For example, in a B-Tree of order 4: `Maximum children = 4` and `Maximum keys = 3`. Consider the following node: `[15 | 30 | 45]`. Once another key is inserted, the node becomes full and must be split.
- **Internal nodes have a minimum capacity**
 - Except for the root and leaf nodes: `Minimum children = N/2` and `Minimum keys = N/2 - 1`
 - This rule prevents the tree from becoming unbalanced. Every internal node must remain sufficiently occupied.
- **All leaf nodes are on the same level**
 - Consider the following tree :



- All leaf nodes appear at exactly the same depth. There are no branches that extend deeper than others.

- This property guarantees that the tree remains balanced, ensuring efficient searching, insertion, and deletion operations.

One way to think about a B-Tree is this: **A Binary Search Tree tries to organize data by creating more levels. A B-Tree tries to organize data by storing more information inside each level.**

As datasets grow from thousands of records to millions, that small idea becomes one of the most important innovations in modern databases and file systems.

2. Operations on B-Tree

One of the most important properties of a B-Tree is that it always remains balanced. Unlike a standard Binary Search Tree, where the shape depends heavily on the insertion order, a B-Tree continuously reorganizes itself whenever a node becomes too full or too empty. This guarantees that all leaf nodes remain at the same level.

As a result, operations such as searching, insertion, and deletion, all have a time complexity of: $O(\log n)$, regardless of the order in which the data is inserted.

For this example, consider the following array:

```
[9, 0, 8, 1, 7, 2, 6, 3, 5, 4]
```

We will construct a **B-Tree of order 3**. For a B-Tree of order 3:

- Maximum children = 3
- Maximum keys = 2

Whenever a node contains more than 2 keys, the middle key is promoted upward and the remaining keys are split into separate nodes.

- **Insert 9** - The tree begins with a single root node.

```
[9]
```

- **Insert 0** - After sorting, the node still satisfies the maximum-key constraint.

```
[0 | 9]
```

- **Insert 8** - After insertion, the node now contains 3 keys. Since **Maximum allowed = 2**, the node must be split.

```
[0 | 8 | 9]
```

The middle key (8) is promoted to become the new root:

```

      [8]
     /  \
    [0]  [9]

```

- **Insert 1** - Since $1 < 8$, we move to the left child. No reorganization is required.

After insertion:

```

      [8]
     /  \
    [0 | 1] [9]

```

- **Insert 7** - Since $7 < 8$, insert into the left child. Since the node exceeds the limit, promote the middle key:

```
[0 | 1 | 7]
```

The tree becomes:

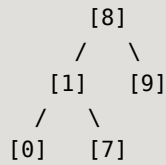
```

      [1 | 8]
     /  |  \
    [0] [7] [9]

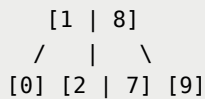
```



Notice that all leaf nodes remain at the same level. A structure such as the following would be incorrect because it would violate the balancing rules of the B-Tree.:



- **Insert 2** - Since: $1 < 2 < 8$, insert into the middle child. No split is required.



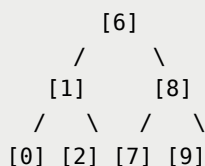
- **Insert 6** - Insert into the middle child node `[2 | 7]`

After sorting:

`[2 | 6 | 7]`

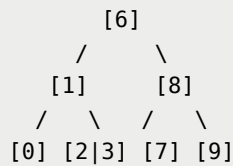
The node exceeds the limit → Promote 6. The root becomes: `[1 | 6 | 8]`.

But the root now also exceeds the limit. We must split again by promoting 6 to become the new root. The tree is balanced once again.



- **Insert 3** - Since: $3 < 6$, move left. Since: $3 > 1$, move to node [2]. No split is necessary.

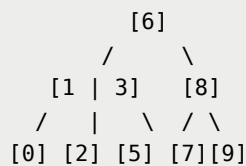
After insertion:



- **Insert 5** - Insert into: [2|3] → After sorting: [2 | 3 | 5]

The node exceeds the limit → Promote 3. The parent becomes: [1|3]

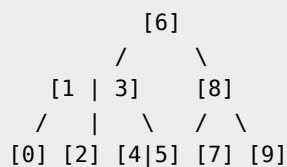
Updated tree:



- **Insert 4** - Since: $4 < 6$, move left. Since: $4 > 3$, move to: [5]

Insert and reorder → [4 | 5]

Final B-Tree:



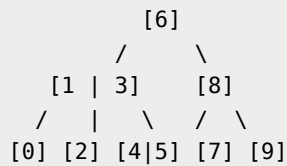
The B-Tree at this point has been successfully formed.

Deletion in B-Tree

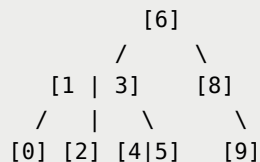
Deletion follows the same general idea as searching and insertion. First, we locate the key to remove.

- **Case 1: Key is in a leaf node**

This is the simplest situation. Suppose we delete 7 from:



The result is:



If the node still satisfies the minimum-key requirement, no reorganization is needed.

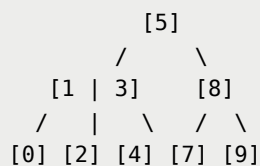
- **Case 2: Key is in an internal node**

Suppose we want to delete 6 from the root. We cannot simply remove it because doing so would disconnect the ordering structure of the tree.

Instead, we replace it with either its predecessor (largest key in the left subtree), or its successor (smallest key in the right subtree)

For example: **Predecessor of 6 = 5** and **Successor of 6 = 7**

Using the predecessor:



After the replacement, the original predecessor key is removed from its leaf node, and the tree is adjusted if necessary.

This additional reorganization is what makes deletion in a B-Tree more complex than insertion. While insertion mainly deals with splitting overflowing nodes,

deletion may require borrowing keys from siblings or merging nodes to maintain the balancing rules of the B-Tree.

3. Implementation of B-Tree

Now that we understand how a B-Tree organizes and balances itself, we can implement it in Python. The implementation consists of two main classes:

- **BNode** → Represents a single node in the B-Tree.
- **BTree** → Represents the entire tree and manages operations such as searching and insertion.

Class BNode

A B-Tree node stores:

```
class BNode:
    def __init__(self, leaf=False):
        self.is_leaf = leaf
        self.keys = []
        self.child = []
```

- `is_leaf`: Indicates whether the node is a leaf
- `keys`: Stores the sorted keys in the node
- `child`: Stores references to child nodes

Display all keys stored in the current node.

```
def display(self):
    print("|", self.keys, "|")
```

Insert a key into a node that still has available space. In other words, insert a key while preserving the B-Tree properties.

```

def insert_non_full(self, key):
    i = len(self.keys) - 1

    if self.is_leaf:
        self.keys.append(None)
        while i >= 0 and key < self.keys[i]:
            self.keys[i + 1] = self.keys[i]
            i -= 1
        self.keys[i + 1] = key

    else:
        while i >= 0 and key < self.keys[i]:
            i -= 1
        i += 1
        if len(self.child[i].keys) == max_keys:
            self.split_child(self.child[i], i)
            if key > self.keys[i]:
                i += 1
        self.child[i].insert_non_full(key)

```

- **If the node is a leaf:** Creates an empty position. Shifts larger keys to the right. Places the new key in sorted order.
- **If the node is not a leaf:** Finds the correct child. Checks whether the child is full. Splits the child if necessary.
- Continues insertion recursively.

Split a full child node into two smaller nodes. Here, we divide a full node into two nodes and promote the middle key.

```

def split_child(self, x, i):
    median = int(max_keys / 2)
    z = BNode(x.is_leaf)

    self.child.insert(i + 1, z)
    self.keys.insert(i, x.keys[median])
    z.keys = x.keys[median + 1: max_keys]

```

```

x.keys = x.keys[0: median]

if not x.is_leaf:
    z.child = x.child[median + 1: max_keys]
    x.child = x.child[0: median + 1]

```

Main Steps

- Find the middle key.
- Create a new right node.
- Move the middle key to the parent.
- Assign keys to the new right node.
- Keep left-side keys.
- Update child pointers if needed.

Create an empty B-Tree with a root node.

```

class BTree:
    def __init__(self):
        self.root = BNode(leaf=True)

```

Start the searching process from the root.

```

def search(self, key):
    self.search_tree(key, self.root)

```

Recursively locate a key inside the B-Tree.

```

def search_tree(self, key, current=None):
    if current is not None:
        n = len(current.keys)
        i = 0

```

```

        while i < n and key > current.keys[i]:
            i += 1
        if i < n and key == current.keys[i]:
            return current, i
        elif current.is_leaf:
            return None
        else:
            return self.search_tree(key, current.child
[i])
    else:
        return None

```

Main Steps

- Find the possible position. If the key is found, return node and position.
- Search ends if a leaf is reached.
- Continue recursively.

Insert a key into the B-Tree.

```

def insert(self, key):
    current = self.root
    if len(current.keys) == max_keys:
        temp = BNode()
        self.root = temp
        temp.child.insert(0, current)
        temp.split_child(temp.child[0], 0)
        temp.insert_non_full(key)
    else:
        current.insert_non_full(key)

```

- If the root is full → Create a new root. Attach the old root. Split the old root. Insert normally afterward.
- **If the root is not full** → Insert directly.

Display the hierarchical structure of the B-Tree. This allows us to visualize the tree level by level.

```
def print_tree(self, node=None, level=0):
    if node is None:
        node = self.root
    print("Level", level, ":", node.keys)
    if not node.is_leaf:
        for child in node.child:
            self.print_tree(child, level + 1)
```

Example Output:

```
Level 0 : [6]
Level 1 : [1, 3]
Level 1 : [8]
Level 2 : [0]
Level 2 : [2]
Level 2 : [4, 5]
Level 2 : [7]
Level 2 : [9]
```

Visit all nodes using Breadth-First Search (BFS), implemented using a circular queue.

```
def traverse(self):
    current = self.root
    if current is None:
        print("Empty Tree")
        return

    nodes_queue = CircularQueue()
    nodes_queue.enqueue(current)
    while not nodes_queue.is_empty():
        node = nodes_queue.dequeue()
        node.display()
```

```
for child in node.child:
    nodes_queue.enqueue(child)
```

Below is the full implementation of B-Tree, including the Circular Queue object at the beginning

[b_tree.py](#)

B-Tree Visualization

<https://www.cs.usfca.edu/~galles/visualization/BTree.html>

Summary

Throughout this section, we explored how B-Trees solve one of the major limitations of traditional Binary Search Trees: maintaining efficiency when storing and managing massive amounts of data.

Unlike Binary Search Trees, where each node can have at most two children, B-Trees allow multiple keys and multiple child nodes within a single node. This significantly reduces the height of the tree, resulting in fewer traversals and fewer disk accesses when performing operations on large datasets.

- We examined the fundamental properties of B-Trees, including their balanced structure, ordered keys, node capacity constraints, and the requirement that all leaf nodes remain at the same level. These properties guarantee that the tree remains balanced regardless of the insertion order.
- We then studied the core operations of B-Trees. During insertion, nodes automatically reorganize themselves through splitting whenever the maximum number of keys is exceeded. During deletion, keys may be replaced by their predecessor or successor to preserve the ordering properties of the tree. Through these mechanisms, the B-Tree continuously maintains its balanced structure without requiring separate balancing operations.

- Finally, we implemented the B-Tree using the **BNode** and **BTree** classes. The implementation demonstrated how searching, insertion, traversal, and node splitting work together to maintain an efficient and scalable data structure.

As a result, B-Trees provide predictable performance for searching, insertion, and deletion operations, all with a time complexity of: $O(\log n)$

This combination of balance, scalability, and efficient disk access makes B-Trees one of the most important data structures in modern computing, forming the foundation of many database management systems, indexing engines, and file systems used today.

References

https://drive.google.com/file/d/1tOqiD1hwn4q_ZbZHeOp-e3oeJfgTEdDq/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/introduction-of-b-tree-2/>

<https://www.youtube.com/watch?v=aZjYr87r1b8>

Next Section:

 [12. Priority Queues](#)